13.1 Concurrency Control: https://www.postgresql.org/docs/current/mvcc-intro.html
13.2 Transaction Isolation: https://www.postgresql.org/docs/current/transaction-iso.html

==========

=== Explicit Locking ===

13.3 Explicit locking: https://www.postgresql.org/docs/current/explicit-locking.html

LOCK: https://www.postgresql.org/docs/current/sql-lock.html

```
LOCK [ TABLE ] [ ONLY ] name [ * ] [, ...] [ IN lockmode
MODE ] [ NOWAIT ]

where lockmode is one of:

    ACCESS SHARE | ROW SHARE | ROW EXCLUSIVE | SHARE
UPDATE EXCLUSIVE
    | SHARE | SHARE ROW EXCLUSIVE | EXCLUSIVE | ACCESS
EXCLUSIVE
```

Table level locks
13.3.1: Table-Level Locks: https://www.postgresql.org/docs/current/explicit-locking.html#LOCKING-TABLES

	AC.SH	**RW SH**	**RW EX**		
SH.U.EX	**SH**	**SH.RW.EX**	**EX**	**AC.EX**	
ACCESS SHARE				x	SELECT
ROW SHARE			x	x	SELECT ... FOR (UPDATE/NO KEY UPDATE/SHARE/KEY SHARE)
ROW EXCLUSIVE		x	x	x	UPDATE/DELETE/ INSERT/MERGE NOTE: **Row-level locks** for UPDATE/DELETE too
SHARE UPDATE EXCLUSIVE	x	x	x	x	VACUUM (=self conflict)
SHARE			x	x	CREATE INDEX
SHARE ROW			x	x	

x	X	x	x	CREATE TRIGGER EXCLUSIVE ALTER TABLE ...
EXCLUSIVE			x	x x x x X x REFRESH MATERIALIZED VIEW CONCURRENTLY
ACCESS EXCLUSIVE		x	x	x x x x x X DROP TABLE, ALTER TABLE/INDEX, TRUNCATE REINDEX, VACUUM FULL, REFRESH MATERIALIZED VIEW

Table level lock
Lock on the **whole table**.
Each **lock type** represents a **group** of **SQL commands**
Locks are either:

a) **non-conflicting** allow **concurrent** usage
i.e. allow SQL commands by **multiple transactions (~ clients)**.

b) **conflicting** only **one** at a time
i.e. only one SQL command at a time

Special case: whether a lock conflicts with **itself**.

a) **non-self-conflicting** allow concurrent execution of **same command**
e.g. **SELECT**

b) **self-conflicting** only one execution of same command
e.g. **ALTER TABLE**

e.g.

Client 1	**Client 2**	**Concurrent?**
SELECT ... FROM a (ROW SHARE)	SELECT ... FROM a (ROW SHARE)	OK
SELECT ... FROM a (ROW SHARE)	UPDATE a ... (ROW EXCLUSIVE)	OK
SELECT .. FROM a (ROW SHARE)	ALTER TABLE a ... (ACCESS EXCLUSIVE)	NO
UPDATE a ... (ROW EXCLUSIVE)	UPDATE a .... (ROW EXCLUSIVE)	OK if **non-conflicting rows** (row-level lock)
VACUUM (SHARE UP. EXC.)	VACUUM (SHARE UP. EXC.)	NO. **Self conflicting**.

Lock acquire/release

Lock acquire: Either

a) **implicit** by the SQL command, or
b) **explicit** with **LOCK** command

Lock release: Either
a) **COMMIT**
b) **ROLLBACK**
c) **SAVEPOINT**

On conflict: Either
a) **WAIT (default)**: wait until lock is released
b) **NOWAIT**: don't wait and throw **error**

e.g. implicit vs explicit

```
-- a) implicit
SELECT * FROM a;                    -- ROW SHARE on a

UPDATE a SET ...;                   -- ROW EXCLUSIVE on a


-- b) explicit
LOCK TABLE a IN ROW EXCLUSIVE;
```

Row level lock
13.3.2: Row-Level Locks: https://www.postgresql.org/docs/current/explicit-locking.html#LOCKING-ROWS

Row level lock
Row-level locks are locks for commands that **modify rows**.
i.e. **DELETE** / **UPDATE** commands.
Row level locks are further locks on top of **table-level locks**.
e.g.
UPDATE a SET command
-> ROW EXCLUSIVE acquire table-level lock for a
-> FOR UPDATE acquire row-level lock for updated row

SQL command	**UPD**	**NO KEY UPD**	**SH**	**KEY SH**
FOR UPDATE DELETE, UPDATE ... (only columns of FOREIGN KEY)	x	x	x	x
FOR NO KEY UPDATE UPDATE ...		x	x	x
FOR SHARE			x	x
FOR KEY SHARE				x

e.g.

```
-- a) implicit
UPDATE a SET a.value = 1;      -- FOR UPDATE
```

```
-- b) explicit
SELECT ... FROM a             -- FOR UPDATE
FOR UPDATE;
```

==========
======

```
This
is
code
Monaco 14
```